Course Title: Visual Programming with C#

Course No.: ICT. Ed. 465
Level: Bachelor
Semester: Sixth

Nature of course: Theoretical + Practical
Credit Hour: 3 hours (2T+1P)
Teaching Hour: 80 hours (32+48)

Practical Work

1. **Setting Up Entity Framework with Code First Approach**
   - **Task:** Create a simple console application using the Code First approach in Entity Framework. Define two entities: Student and Course. The Student entity should have properties such as StudentID, Name, and EnrollmentDate. The Course entity should have CourseID, Title, and Credits.
   - **Challenge:** Implement a one-to-many relationship between Student and Course, where a student can enroll in multiple courses. Then, use migrations to create the database and tables in SQL Server.
2. **Querying Data using LINQ**
   - **Task:** Using the database created in the previous exercise, write a LINQ query to retrieve the list of all students who have enrolled in a specific course (e.g., "Mathematics").
   - **Challenge:** Display the student's name, enrollment date, and the course title. Ensure the query handles cases where no students are enrolled in the course.
3. **Database First Approach in Entity Framework**
   - **Task:** Reverse-engineer an existing SQL Server database using the Database First approach. The database should contain tables for Employees and Departments.
   - **Challenge:** Generate the entity models and context classes, and then write a LINQ query to retrieve all employees working in a particular department (e.g., "HR"). Display the employee's name and department.
4. **Implementing CRUD Operations with Entity Framework**
   - **Task:** Create a console or web application using the Code First or Database First approach in Entity Framework. Implement basic CRUD operations (Create, Read, Update, Delete) for managing a Product entity with properties such as ProductID, Name, Price, and Category.
   - **Challenge:** Ensure that you handle exceptions, such as trying to delete a product that doesn't exist in the database, and provide meaningful error messages.
5. **Combining LINQ Queries with Entity Framework**
   - **Task:** Using the previously created Student and Course entities, write a complex LINQ query that retrieves all students who enrolled in courses within a specific date range (e.g., between January 1, 2023, and December 31, 2023).
   - **Challenge:** Further, group the results by course title and display the count of students enrolled in each course.
6. **Understanding and Using Delegates**
   - **Task:** Create a console application that demonstrates the use of delegates. Define a delegate named MathOperation that can point to methods performing different arithmetic operations (e.g., addition, subtraction, multiplication, division).

- **Challenge:** Create methods for each arithmetic operation and use the delegate to invoke these methods. Allow the user to input two numbers and choose an operation, and then display the result using the delegate.

7. **Lambda Expressions in Action**
   - **Task:** Using a list of integers, write a program that uses lambda expressions to perform the following operations:
     Find all even numbers.
     Calculate the square of each number.
     Filter numbers greater than a certain value (e.g., 50).
   - **Challenge:** Implement these operations using LINQ methods (Where, Select, etc.) and lambda expressions.
8. **Event Handling in C#**
   - **Task:** Create a simple event handling system using C#. Define an event called ThresholdReached that triggers when a counter exceeds a certain value.
   - **Challenge:** Create a class with methods to increment the counter and check if the threshold is reached. When the event is triggered, display a message indicating the threshold has been exceeded. Demonstrate this functionality in a console application.
9. **String Manipulation and StringBuilder**
   - **Task:** Write a program that takes a sentence as input and performs the following string manipulations:

     Count the number of words in the sentence.
     Reverse the entire sentence.
     Convert the sentence to uppercase.
   - **Challenge**: Use StringBuilder to efficiently build and manipulate a large string, such as appending 1000 times the reversed version of the input sentence.
10. **Working with Collections: Generic and Non-Generic**
    - **Task:** Create a console application that demonstrates the use of both generic and non-generic collections. Use a List<T> to store a collection of student names (strings) and a non-generic ArrayList to store a mixed collection of different data types (e.g., integers, strings, and objects).
    - **Challenge:** Perform the following operations:
      Add, remove, and search for elements in both collections.
      Sort the List<T> alphabetically.
      Handle type casting when retrieving elements from the ArrayList.

11. **Working with Data Types, Variables, and Operators**
    - **Task:** Write a C# console application that takes input from the user, such as their name, age, and salary. Perform the following operations:
      Calculate the user's age in months.
      Calculate a 10% bonus based on the user's salary.
      Determine whether the salary is greater than a specific threshold (e.g., $50,000) using conditional operators.
    - **Challenge:** Handle different data types, such as int, double, and string, and display the results in a formatted manner.

12. **Implementing Control Statements and Arrays**
    - **Task:** Create a program that allows the user to input 10 integers and store them in an array. Implement the following functionality using control statements:
      Find and display the maximum and minimum values in the array.
      Sort the array in ascending order.
      Check if a specific number exists in the array using a loop.
    - **Challenge:** Write a method to calculate the average of the numbers in the array and display it.
13. **Working with Classes, Structures, and Enumerations**
    - Task: Define a Student class with properties such as ID, Name, and Grade. Implement a method to calculate the final grade based on assignment scores (e.g., use an array to store scores). Additionally, define a GradeLevel enumeration with values such as A, B, C, etc.
    - Challenge: Create a StudentRecord structure that includes the student's class information, and then demonstrate how to use the class, structure, and enumeration in a console application.
14. **Exploring Partial, Static, and Sealed Classes**
    - **Task:** Create a C# project that includes the following:
      A Calculator class split across two partial class files. Implement methods like Add, Subtract, and Multiply in one file, and Divide, Square, and SquareRoot in another.
      A static Utility class with methods such as ConvertToUpperCase and CalculateAreaOfCircle.
      A sealed class FinalReport that cannot be inherited, with a method to display a summary report.
    - **Challenge:** Create an instance of Calculator, call its methods, and use the static Utility class without instantiation. Demonstrate the behavior of the sealed class.
15. **Implementing Inheritance, Polymorphism, Interfaces, and Exception Handling**
    - **Task**: Create a base class Animal with methods such as MakeSound. Derive two subclasses, Dog and Cat, that override the MakeSound method. Implement polymorphism by calling the overridden methods through a base class reference.
    - **Challenge:** Define an interface IMovable with a method Move. Implement this interface in the Animal class and demonstrate the use of interface methods. Additionally, add exception handling to catch invalid operations, such as calling Move when the object is null.
16. **Understanding .NET Framework, .NET Core, and .NET Standard**
    - **Task:** Research and compare the .NET Framework, .NET Core, and .NET Standard in terms of their features, use cases, and platform compatibility. Create a document or presentation summarizing the differences and scenarios where each would be most appropriate.
    - **Challenge:** Provide examples of real-world applications that would benefit from each platform. Explain why you would choose one over the others for a specific project (e.g., a web application, cross-platform mobile app, or desktop application).
17. **Exploring .NET Components: Common Language Runtime (CLR) and Class Library**
    - **Task:** Create a simple C# console application that demonstrates how the Common Language Runtime (CLR) manages memory and handles exceptions. Write a few lines of code that intentionally cause an exception (e.g., dividing by zero), and observe how the CLR handles it.
    - **Challenge:** Explore and use some classes from the .NET Class Library, such as System.IO for file handling, or System.Collections.Generic for working with collections. Write code snippets demonstrating their usage and explain how the CLR interacts with these classes.

18. **Setting Up Visual Studio and Exploring Project Types**
    - **Task:** Set up Visual Studio or Visual Studio Code as your development environment. Create different types of projects available in .NET, such as a Console App, a Class Library, and a Web API project. Familiarize yourself with IntelliSense, debugging tools, and other IDE features.
    - **Challenge:** Write a brief report or create a video walkthrough showing how to create and configure each project type, along with a simple "Hello World" program. Explain the key differences between these project types and their typical use cases.

19. **Comparing ASP.NET vs ASP.NET Core**
    - **Task:** Research and compare ASP.NET and ASP.NET Core in terms of performance, cross-platform support, and flexibility. Create a comparison document or presentation outlining the key differences and advantages of each framework.
    - **Challenge:** Based on your comparison, choose one of the frameworks and justify why it would be the better choice for developing a large-scale enterprise web application. Include factors like scalability, deployment options, and long-term maintenance.
20. **Creating a Simple Web Forms Application**
    - Task: Create a basic ASP.NET Web Forms application that allows users to submit a form with their name, email, and message. The application should display a confirmation message after form submission.
    - Challenge: Add validation to the form fields (e.g., ensuring the email is in the correct format and all fields are required). Use ASP.NET Web Forms controls like TextBox, Button, and Label to accomplish this.
21. **Building an ASP.NET MVC Application with Models, Views, and Controllers**
    - **Task:** Create a simple ASP.NET MVC application for managing a list of books. The application should allow users to add, edit, and delete books, with each book having properties such as Title, Author, and PublishedYear.
    - **Challenge:** Implement the MVC architecture by creating models, views, and controllers for the Book entity. Use URL routing to map user requests to the appropriate controller actions, and ensure that the application follows the RESTful convention for handling CRUD operations.
22. **Implementing Razor Syntax and Creating a Layout for ASP.NET Web Pages**
    - **Task:** Design a basic layout for an ASP.NET Web Pages application using Razor syntax. The layout should include a header, footer, and navigation menu that appears on all pages.
    - **Challenge:** Create a few content pages that use this layout, such as a home page and an about page. In the Razor code, demonstrate the use of variables, loops, and conditional expressions (e.g., displaying different messages based on user input or time of day).
23. **Database Interaction in ASP.NET MVC**
    - **Task:** Extend the ASP.NET MVC application created in question 3 to interact with a database. Use Entity Framework to create a database context and perform CRUD operations on the Book entity.
    - **Challenge:** Implement data validation on the model (e.g., ensuring that the Title field is required and the PublishedYear is a valid year). Display appropriate error messages in the views when the validation fails, and ensure that the database interactions are efficient and secure.